Rapport

28 juin 2026

Table des matières

1 Introduction **1**

2 Définitions et hypothèses **2**

3 Évaluation d'un contexte causal contrefactuel **3**

3.1 Fonction de coût 3

3.2 Distance à l'observation : coûts des opérations 4

3.3 Vraisemblance du contrefactuel 6

3.3.1 Juge expert 6

3.3.2 Construction d'une politique optimale à partir de l'expert 6

3.3.3 Évaluation de la vraisemblance à partir de la politique optimale 8

3.3.4 Exploitation directe de l'expert 9

3.3.5 Sélection des événements à évaluer 12

3.4 Critère de validité 13

4 Transition d'un framework RAC vers LTS : PDDL **13**

4.1 Rappel PDDL 13

4.2 Construction d'un LTS à partir d'un problème PDDL 13

4.3 Remarques et limites 15

4.4 Exemple PDDL 16

5 Application : exemple du poison **16**

5.1 De PDDL vers un LTS 17

5.2 Définition des contextes causaux 18

5.3 Définition des politiques optimales 20

5.4 Expérimentations 21

A Exemple contexte causal non valide pour la fonction de coût **24**

A.1 Fonction *divergence* 24

A.2 Fonction *correct* 24

B Construction d'une politique optimale avec planification et l'utilisation de l'algorithme A* **24**

C Vérification des propriétés pour les opérateurs **25**

C.1 Opérateur géométrique 25

C.2 Opérateur gamma 26

C.3 Opérateur de seuil 26

C.4 Synthèse résultats 27

D Digression sur le concept de sélection causale dans le domaine des sciences cognitives **28**
D.1 Mutabilité et propensité des facteurs 28
D.2 Impact des normes sur le raisonnement contrefactuel 28
D.3 Impact du temps sur le raisonnement contrefactuel 29

E Démonstrations des propriétés du parser **29**
E.1 Correction 30
E.2 Terminaison 31
E.3 Complétude 31

F PDDL **32**
F.1 Domaine Block World classique 32
F.2 Domaine et problème Poison 33

1 Introduction

De nombreuses situations peuvent être décrites comme des séquences d'états et d'événements qui produisent un résultat particulier. Pour mieux comprendre les résultats de ces scénarios, il est assez naturel de s'interroger sur ce qui aurait pu se produire autrement. Et si une décision avait été différente ? Et si ce facteur n'avait pas été présent ? Ce type de réflexion, correspondant à envisager des scénarios alternatifs à une réalité que l'on observe, est au centre du raisonnement contrefactuel. Cette notion ne se limite pas à la réflexion humaine, elle occupe une place centrale dans différents domaines de recherche tels que la causalité actuelle ou l'Explainable Artificial Intelligence (XAI).

En XAI, on utilise le raisonnement contrefactuel pour produire une explication, qui prend la forme d'un changement minimal à effectuer sur une entrée permettant de changer la sortie d'un système. Cette notion de changement minimal est contrainte et définie selon différents critères afin d'obtenir une explication utile et précise. Concernant la causalité actuelle, l'approche contrefactuelle définit une cause comme un fait qui produit une différence. Selon cette perspective introduite par Lewis "c est la cause de e" équivaut à "si c n'avait pas eu lieu, e n'aurait pas eu lieu non plus". On cherche à définir des causes en observant les mondes possibles où la cause est absente.

Nous faisons le lien entre ces deux domaines sur la notion "des mondes possibles", en définissant une fonction de coût permettant d'évaluer leur pertinence relativement à la réalité. Notre travail s'inscrit dans la continuité du cadre introduit dans Sarmiento (2025) qui propose un formalisme général basé sur des systèmes de transitions étiquetés (LTS) correspondant à des graphes orientés. Une trajectoire dans un LTS est donnée par une politique déterministe qui associe à chaque état un événement à exécuter. Nous évaluons des contextes causaux contrefactuels, qui correspondent à des trajectoires alternatives à une trajectoire observée, selon deux axes : leur proximité à l'observation et leur vraisemblance. Sur la question de vraisemblance, ce travail introduit une forme d'acceptabilité des contrefactuelles relative à des normes prescriptives ou statistiques. Pour cela les arêtes des graphes sont augmentées par un attribut de coût relatif à des normes. La fonction de coût proposée est modulaire, ses composantes peuvent être adaptées en fonction des applications. Cette modularité s'appuie notamment sur des travaux en sciences cognitives qui s'intéressent, elles aussi, au raisonnement contrefactuel.

Ce rapport est organisé de la façon suivante : la section 1 reprends et adapte les définitions et hypothèses formelles du cadre proposé par Sarmiento (2025). Puis la section 2 introduit la fonction de coût et sa modularité (choix des différentes composantes, opérateurs, évaluation normative...), enfin, nous concluons cette partie avec l'introduction d'un parseur permettant de passer d'un problème PDDL à notre formalisme dans la partie 4. Dans une seconde partie applicative, nous montrons une pipeline complète d'évaluation sur un exemple concret (exemple du poison). Nous partons d'un problème PDDL que nous convertissons en LTS afin de se placer dans notre framework puis nous expérimentons autour de différentes configurations de la fonction de coût pour mieux en saisir les nuances.

2 Définitions et hypothèses

On définit dans un premier temps l'ensemble des objets nécessaire à l'évaluation des contextes causaux contrefactuels. Une partie des définitions est directement extraite de l'article AMAAS Sarmiento 2025, d'autres ont été légèrement modifiées et certaines ont été rajoutées spécifiquement pour l'étude.

- $\mathbb{F}$: ensemble de fluents qui décrivent l'environnement
- $\mathcal{F}$: ensemble des formules d'état, formules de logique classique construites sur $\mathbb{F}$
- $\mathbb{E}$: ensemble des variables qui décrivent les transitions, "ground events" qui modifient les fluents
- $\mathbb{S} \subseteq 2^{\mathbb{F}}$: ensemble de tous les états possibles du monde (exclusion des états incohérents)
- $\tau \subseteq \mathbb{S} \times 2^{\mathbb{E}} \times \mathbb{S}$: ensemble des relations de transition. Pour chaque état indique les différents ensembles d'événements simultanés qui pourraient se produire ainsi que les états résultants
- $\langle \mathbb{S}, \tau \rangle$: Labelled Transition System, graphe dirigé avec $\mathbb{S}$ les nœuds du graphe et τ les arcs du graphe. *NB* : définition conventionnelle d'un graphe (nœuds + arcs), cette définition permet aussi de garder l'information d'états qui n'ont aucune transition entrante ou sortante. Soit $pre : \mathbb{E} \to \mathcal{F}$, la fonction qui à chaque événement associe des préconditions, on considère dans notre contexte que tous les LTS sont corrects relativement à cette fonction.
- Politique $\pi : \mathbb{S} \to 2^{\mathbb{E}}$ relativement à un LTS donné. À la différence de l'article AAMAS les événements ne dépendent que d'un état et non d'un couple (état, temps). On considère dans notre cadre que les états contiennent l'ensemble des informations nécessaires ce qui peut inclure des informations implicites sur le temps.
- Π_τ l'ensemble de toutes les politiques possibles relativement à un LTS $\langle \mathbb{S}, \tau \rangle$
- Politique correcte π relative à un LTS : $\forall S \in \mathbb{S} \quad \exists S' \in \mathbb{S}, (S, \pi(S), S') \in \tau$
- Contexte causal : $\chi = (\pi, S_0, \tau)$ avec $\pi \in \Pi_\tau$ une politique, $S_0 \in \mathbb{S}$ un état de départ et τ un ensemble de relations de transitions décrivant un LTS.
- Hypothèse cas déterministe : Soit un LTS défini par $\langle \mathbb{S}, \tau \rangle$ est dit déterministe si :

$$\forall S, S' \in \mathbb{S}, \forall E \in 2^{\mathbb{E}}, \quad (S, E, S') \in \tau \implies \nexists S'' \in \mathbb{S}, \quad (S, E, S'') \in \tau$$

- $\mathbb{A} \subseteq \mathbb{S}$: ensemble des états terminaux d'un LTS $\langle \mathbb{S}, \tau \rangle$

$$\mathbb{A} = \{S \mid S \in \mathbb{S},\ (S, \emptyset, S) \in \tau\}$$
$$\text{et} \quad \forall S \notin \mathbb{A},\ \forall E \in 2^{\mathbb{E}},\ (S, E, S) \notin \tau$$

- $\mathbb{T}$: ensemble des pas de temps
- $S^\chi : \mathbb{T} \to \mathbb{S}$ la fonction relative à un causal setting $\chi = (\pi, S_0, \tau)$ qui indique au pas de temps $t \in \mathbb{T}$ l'état dans lequel on se trouve en suivant la politique π, dans τ depuis S_0.

- $E^{\chi} : \mathbb{T} \to \mathbb{S}$ la fonction relative à un causal setting $\chi = (\pi, S_0, \tau)$ qui indique au pas de temps $t \in \mathbb{T}$ l'événement qui a lieu en suivant la politique π, dans τ depuis S_0
- σ^{χ} : avec $\chi = (\pi, S_0, \tau)$, la trace complète (états, événements), obtenue dans le contexte causal χ, de la forme : $[S_0, E_0, S_1, E_1, \ldots, E_{n-1}, S_n, E_n, S_n, E_n \ldots]$. Une fois un état terminal atteint, on observe une boucle qui amène à une trace infinie. On simplifie cette trace pour qu'elle soit de longueur finie. Soit n tel que $S_n \in \mathbb{A}$: la trace d'origine devient $\sigma^{\chi} = [S_0, E_0, S_1, E_1, \ldots, E_{n-1}, S_n]$. On notera $\sigma^{\chi}_{|state}$ la trace ne contenant que les états et, $\sigma^{\chi}_{|event}$ la trace ne contenant que les événements.
- $expert_judge_{\tau} : \mathbb{S} \times 2^{\mathbb{E}} \to [0, 1]$ fonction d'évaluation experte, définie partiellement sur l'ensemble des couples (S, E) tel que $\exists S' \in \mathbb{S}, (S, E, S') \in \tau$. Cette fonction attribue à chaque paire une valeur comprise entre 0 et 1 relative à une norme (statistique ou prescriptive). Ces valeurs sont fixées de manière subjective.

3 Évaluation d'un contexte causal contrefactuel

Pour évaluer un contexte causal contrefactuel relativement à un contexte causal observé, on procède en deux temps. Dans un premier temps, on attribue un coût au contexte causal, ce dernier permet d'évaluer la proximité à l'observation, ainsi que la plausibilité du contexte. Le coût permet aussi d'agir comme un filtre à priori sur les contrefactuels non valides relativement au LTS. Une fois le coût calculé, on applique un critère de validité (au sens XAI), qui permet de vérifier que les sorties obtenues, en appliquant respectivement les deux politiques, à partir de l'état de départ dans le LTS, sont différentes. Ce critère agit comme un filtre à posteriori sur les contextes causaux contrefactuels.

3.1 Fonction de coût

La fonction de coût doit permettre un compromis entre l'évaluation :

- de la distance du contexte causal contrefactuel par rapport au causal setting observé. L'intuition derrière ce terme correspond à l'idée qu'un bon contrefactuel doit être relativement proche du contexte causal observé pour être pertinent
- de la plausibilité du contexte causal. La plausibilité ou encore le réalisme correspond ici à un respect relatif d'une norme (statistique ou prescriptive).

La fonction de coût intègre deux termes supplémentaires qui permettent d'exclure les contrefactuels non valides relativement au LTS (politiques non correctes), ainsi que les politiques qui ne divergent de la politique observée que sur des états non présents dans la trace d'états observée.

$$Coût_{\tau}(\chi, \chi') = \lambda_1 d_obs(\sigma^{\chi}_{|state}, \sigma^{\chi'}_{|state}) + \lambda_2 v(\chi', expert_judge_{\tau}) + divergence(\chi, \chi') + correct(\chi') \tag{1}$$

Avec :

- $\chi = (\pi, S_0, \tau)$ le contexte causal observé et $\chi' = (\pi', S'_0, \tau)$ le contexte causal contrefactuel que l'on évalue.
- *d_obs* attribue un coût relatif à la distance entre le contexte causal contrefactuel par rapport à celui observé. On utilise la distance de Levenshtein, avec variation des poids en fonction de l'intervention effectuée (suppression, insertion, substitution).
- v la fonction de vraisemblance du contexte causal contrefactuel par rapport à un jugement expert.
- La fonction *divergence* permet d'exclure les contextes causaux contrefactuels si leur politique associée ne diverge de celle observée que dans des états non présents dans la trace d'états du contexte causal observé.

$$divergence(\chi, \chi') = \begin{cases} 0 \quad \text{si} \quad \exists t < len(\sigma^{\chi}_{|state}) \quad \pi(S^{\chi}(t)) \neq \pi'(S^{\chi}(t)) \\ +\infty \quad \text{sinon} \end{cases} \tag{2}$$

  Voir exemple A.1

- La fonction *correct* permet d'exclure les contextes causaux contrefactuels dont la politique n'est pas correcte :

$$correct(\chi') = \begin{cases} 0 \quad si\ \pi'\ correcte\ relativement\ au\ LTS\ associé\ à\ \tau \\ +\infty \quad sinon \end{cases} \tag{3}$$

  Voir exemple A.2.

3.2 Distance à l'observation : coûts des opérations

Des travaux en sciences cognitives ont montré l'impact des notions de propensité (vraisemblance d'obtenir un effet sachant qu'un événement se produit) et de mutabilité (facilité avec laquelle on peut imaginer qu'un événement ait été différent) des événements lors du processus de raisonnement contrefactuel pour la sélection causale (voir D.1). Notre cadre est pour l'instant déterministe donc nous laissons de côté la notion de propensité. Mais il reste intéressant d'intégrer la mutabilité, notamment pour le calcul de distance à l'observation.

On souhaite modifier le coût de l'opération de substitution d'un état dans la distance de Levenshtein, c'est-à-dire que lorsque l'on cherche à déterminer le coût de transformation de la trace observée vers la trace contrefactuelle, on souhaite que la substitution d'un événement très peu mutable ait un coût de substitution plus élevé que pour un événement très mutable. Pour cela la première chose à faire est de définir formellement la notion de mutabilité dans notre framework.

Soit un événement E qui se produit dans un état S d'un LTS défini par $< \mathbb{S}, \tau >$ de la façon

suivante :

$$mutabilite(E, S) = \#\{(S, E', S') \in \tau\}. \tag{4}$$

La mutabilité correspond ainsi au nombre d'arcs sortants de l'état de départ d'un événement. Pour un état donné, s'il possède un grand nombre d'arcs sortants, on considère ses événements possibles comme très mutables (on peut facilement imaginer qu'un autre événement ait eu lieu).

La distance de Levenshtein est calculée non pas sur les événements, mais sur les états, on adapte donc cette définition. Le coût de substitution d'un état de la trace observé par un autre état correspond à l'inverse de la mutabilité de l'état le précédent dans la trace observée. Et la mutabilité d'un état correspond à son nombre d'arcs sortants.

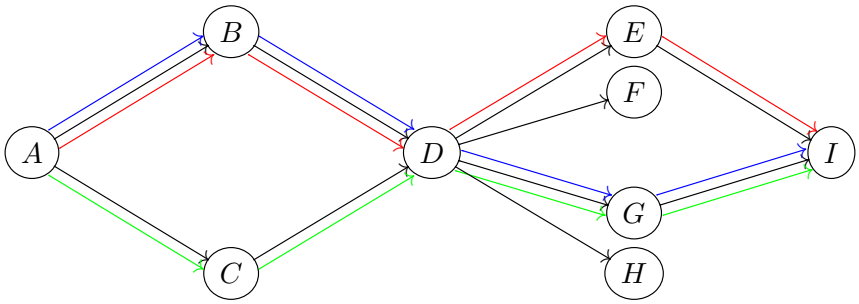


FIGURE 1 – Exemple d'application de la distance de Levenshtein avec coût de substitution dynamique. On considère que la trace bleue [ABDGI] est la trace observée et on cherche à mesurer la distance de la trace rouge [ABDEI] à la trace bleue, ainsi que de la trace verte [ACDGI] à la bleue. Les traces rouge et verte ne diffèrent de la bleue que par un état. Pour la rouge, c'est le choix dans l'état D (mutabilité = 4) qui a amené à l'état E. Pour la verte, c'est le choix dans A (mutabilité = 2) qui a amené à l'état C. La distance entre la trace rouge et la trace bleue est de $\frac{1}{4}$ et celle entre la trace bleue et la trace verte est de $\frac{1}{2}$. La trace verte est donc plus éloignée de la bleue que la trace rouge, car la mutabilité est plus faible en A qu'en D.

On observe dans l'exemple 1 l'intérêt de modifier le coût de substitution en fonction de la mutabilité s'il avait été fixé à un la distance à l'observation pour les traces rouge et bleue aurait été la même. Mais on considère maintenant plus proche une trace contrefactuelle si elle diffère en des états très mutables de la trace observée.

Cette définition et intégration de la mutabilité dans la distance à l'observation reste une première piste et peut présenter de potentielles limitations. En effet, cette définition dépend de la modélisation du monde (le LTS) que nous choisissons de ne pas remettre en question. De plus la question posée par ce nouveau coût de substitution est : "à quel point il est coûteux de remplacer l'événement observé" alors que l'on pourrait plutôt souhaiter répondre à la question "à quel point il est coûteux de remplacer l'événement observé par l'événement spécifique du scénario contrefactuel".

3.3 Vraisemblance du contrefactuel

Pour évaluer la vraisemblance (fonction v) du contexte causal contrefactuel, on se base sur l'évaluation extérieure d'un juge expert, comme définie dans la partie 2. Cette évaluation permet d'attribuer un score compris entre 0 et 1 à chaque couple état/événement d'un LTS. Un score d'un correspond à un respect total de la norme, un score de 0 à un non-respect total de la norme. À partir des scores, on évalue la vraisemblance du contrefactuel soit en l'exploitant directement, soit en construisant à partir de ces derniers une politique optimale.

3.3.1 Juge expert

Différentes méthodes sont applicables pour attribuer les scores. On peut contraindre, par exemple, pour un état donné, à avoir une distribution de probabilité ou de possibilité sur l'ensemble des événements possibles depuis cet état.

1. Distribution de probabilité : L'expert donne une distribution de probabilité sur les arcs sortant d'un état, deux possibilités de construction :
    - À partir d'un classement avec softmax sur l'inverse des positions : simple, mais on perd en expressivité, les écarts entre les événements sont fixes relativement au nombre d'arcs sortants du sommet qui représente un état. De plus cela demande d'avoir un ordre sur les événements ce qui n'est pas toujours souhaitable si on a des événements normativement équivalents.
    - L'expert donne directement une distribution de probabilités, cette méthode a l'avantage de limiter la perte d'expressivité et d'autoriser des scores égaux pour deux événements sortant d'un même état.
    - Une distribution de probabilité peut fortement pénaliser des événements issus d'un état avec de nombreux arcs sortant comme on l'observe sur le LTS ci-dessous :
2. Distribution de possibilité : l'expert attribue une distribution de possibilité sur l'ensemble des arcs sortant d'un état donné. On note qu'une distribution de possibilité impose que le score maximal parmi tous les scores des événements sortant d'un état soit de 1.
3. Valeurs libres : l'expert juge indépendamment chaque événement possible en attribuant une valeur entre 0 et 1. Avantage : pas de contrainte sur la somme des scores ou encore sur leur valeur maximale. Désavantage : interprétation et choix des valeurs potentiellement plus complexe.

3.3.2 Construction d'une politique optimale à partir de l'expert

Pour évaluer la vraisemblance d'un contexte causal contrefactuel à partir d'une évaluation experte, on peut premièrement construire une politique optimale vis-à-vis de l'évaluation puis dans un second temps comparer nos deux politiques (optimale et contrefactuelle). La construction de politiques optimales peut être contrainte selon l'application, en fonction d'une heuristique, en prenant

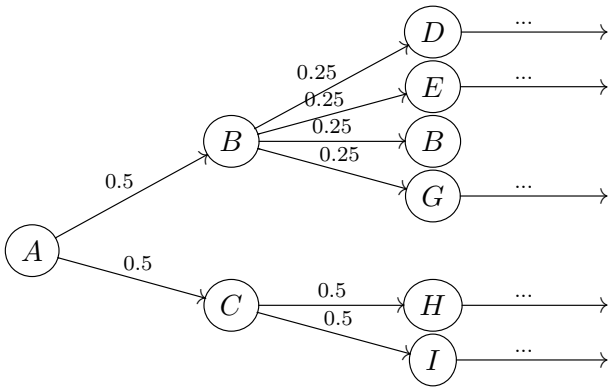


FIGURE 2 – Exemple de LTS contraint à des distributions de probabilités. Pour les états B et C chaque événement sortant est normativement équivalent aux autres. Les arcs sortant de l'état B sont pénalisés par leur nombre.

en compte le coût de calcul de la construction, etc. Les politiques optimales dépendent uniquement d'un LTS et de l'évaluation d'un juge expert, elles sont indépendantes des contextes causaux contrefactuels qui sont évalués.

1. Politique optimale sans planification : pour chaque état cette politique indique l'événement qui maximise l'évaluation de l'expert à un horizon temporel d'un seul pas de temps. La fonction $\pi_\tau^{opt} \in \Pi_\tau$ est définie par :

$$\forall S \in \mathbb{S} \quad \pi_\tau^{opt}(S) = \underset{E}{\operatorname{argmax}}(expert_judge_\tau(S, E)) \tag{5}$$

   Cette politique a pour avantage sa simplicité de construction, d'interprétation ainsi que son coût de calcul. Dans certains cas, elle possède tout de même certaines limitations comme on l'observe dans l'exemple ci-dessous :

   La politique optimale sans planification associée au LTS de la Figure 3 corresponds à :

$$\begin{aligned} &\pi_\tau^{opt1}(S_0) = a, \quad \pi_\tau^{opt1}(S_1) = a, \quad \pi_\tau^{opt1}(S_2) = b, \\ &\forall i \in [3,4,5,6] \quad \pi_\tau^{opt1}(S_i) = \emptyset \end{aligned} \tag{6}$$

   Si on fixe l'état de départ à S_0 on obtient, en suivant la politique optimale, la trace : $[S_0, a, S_1, a, S_3]$ qui correspond au chemin rouge. La moyenne arithmétique des scores des événements de cette trace vaut 0.45 (le choix de la moyenne arithmétique est discutable). Cependant, on peut considérer dans ce cas que la bonne décision à prendre, à partir de l'état S_0, en connaissant le LTS et les scores qui y sont associés est l'événement b. En effet, on obtient la trace $[S_0, b, S_2, b, S_6]$, correspondant au chemin vert et qui a une moyenne arithmétique sur les scores des événements de la trace de 0.65.

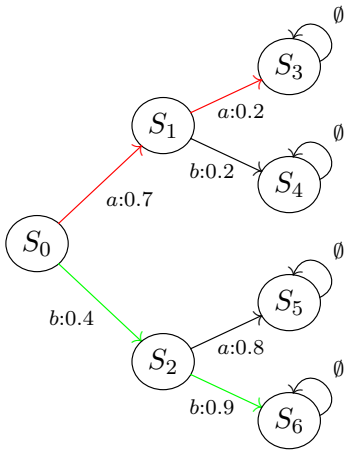


FIGURE 3 – Exemple de LTS évalué par un expert qui représente un cas limite pour une politique sans planification. Étiquetage des arcs du graphe de la forme "événement :score"

De cet exemple, on tire deux observations. La politique optimale sans planification ne tire pas parti de la connaissance totale du LTS et des scores associés aux événements. Deuxièmement, on note que la politique optimale n'est pas unique. En effet, un choix arbitraire a été effectué en S_1 pour le choix de l'événement a. Dans les cas où plusieurs événements maximisent le score, on peut appliquer une seconde heuristique ou choisir aléatoirement.

2. Politique optimale avec planification : on peut généraliser la définition 5 à un horizon temporel n. Pour cela, dans chaque état, on applique un algorithme du plus court chemin dans un graphe pondéré (par exemple A*) et on sélectionne le premier événement de ce chemin. Voir B pour plus de détail sur la construction de cette politique. Ce type de politique optimale permet de prendre en compte la connaissance totale du LTS et en fonction de l'algorithme de plus court chemin, on peut limiter l'augmentation du coût de construction de la politique. Une telle politique permet de mieux gérer les cas du même type que la Figure 3. On obtient, par exemple pour le LTS de la figure 3 une politique de la forme :

$$\pi_\tau^{opt2}(S_0) = b, \quad \pi_\tau^{opt2}(S_1) = a, \quad \pi_\tau^{opt2}(S_2) = b, \quad \forall i \in [3,4,5,6] \quad \pi_\tau^{opt2}(S_i) = \emptyset \tag{7}$$

Comme pour les politiques optimales sans planification, il n'y a pas unicité du résultat. Un traitement spécifique est nécessaire pour les états qui possèdent plusieurs événements qui maximise le score.

3.3.3 Évaluation de la vraisemblance à partir de la politique optimale

À partir d'une politique optimale donnée, on peut définir un coût de vraisemblance entre cette dernière et la politique associé à un contexte causal contrefactuel. On définit une fonction de distance

entre deux politiques sur une trace d'état :

$$dist_politiques(\pi, \pi', \sigma_{|state}) = \frac{\#\{\pi(S) \neq \pi'(S) | S \in \sigma_{|state}\}}{\#\{S \in \sigma_{|state}\}}. \tag{8}$$

On peut utiliser cette distance pour évaluer la vraisemblance entre une politique associée à un contexte causal contrefactuel $\chi' = (\pi', S_0, \tau)$ et une politique optimale π_τ^{opt} construite à partir d'une évaluation experte $expert_judge_\tau$:

$$v(\chi', expert_judge_\tau) = dist_politiques(\pi', \pi_\tau^{opt}, \sigma_{|state}^{\chi'}) \tag{9}$$

On compte le nombre de fois où la politique contrefactuelle n'a pas effectué un choix optimal sur le chemin (la trace) obtenue dans son contexte causal.

Exemple : Soit le LTS de la Figure 3 $\langle \$, \tau \rangle$ on définit un contexte causal $\chi' = (\pi', S_0, \tau)$ avec :

$$\pi'(S_0) = b, \quad \pi'(S_1) = b, \quad \pi'(S_2) = a, \quad \forall i \in [3,4,5,6] \quad \pi'(S_i) = \emptyset$$

On obtient la trace $\sigma^{\chi'} = [S_0, b, S_2, a, S_5]$ et donc $\sigma_{|state}^{\chi'} = [S_0, S_2, S_5]$

- Coût de vraisemblance avec π_τ^{opt1} :

$$dist_politiques(\pi', \pi_\tau^{opt1}, \sigma_{|state}^{\chi'}) = \frac{\#\{S_0, S_2\}}{\#\{S_0, S_2, S_5\}} = \frac{2}{3}$$

- Coût de vraisemblance avec π_τ^{opt2} :

$$dist_politiques(\pi', \pi_\tau^{opt2}, \sigma_{|state}^{\chi'}) = \frac{\#\{S_2\}}{\#\{S_0, S_2, S_5\}} = \frac{1}{3}$$

On observe que le résultat du coût de vraisemblance dépends de la politique optimale même si l'évaluation experte sur lesquelles π_τ^{opt1} et π_τ^{opt2} sont construites est la même.

3.3.4 Exploitation directe de l'expert

Une autre approche possible pour évaluer la vraisemblance d'un contexte causal contrefactuel est de directement exploiter l'évaluation experte, en agrégeant les scores des événements associés à la trace du contexte causal contrefactuel. Pour cela différents opérateurs d'agrégation peuvent être utilisés en fonction de l'application.

Soit Agg un opérateur d'agrégation, un LTS $\langle \$, \tau \rangle$, un contexte causal contrefactuel $\chi' = (\pi', S_0, \tau)$ avec $\sigma^{\chi'} = [S^{\chi'}(0), E^{\chi'}(0), \ldots, E^{\chi'}(n-1), S^{\chi'}(n)]$ et $expert_judge_\tau$ une évaluation experte sur le LTS, la fonction v de l'équation 1 est de la forme :

$$
\begin{aligned}
&v(\chi', expert_judge_\tau) = 1 - Agg(s_0, \ldots, s_{n-1}) \\
&\forall i < len(\sigma^{\chi'}_{|state}) \quad s_i = expert_judge(S^{\chi'}(i), E^{\chi'}(i))
\end{aligned}
\tag{10}
$$

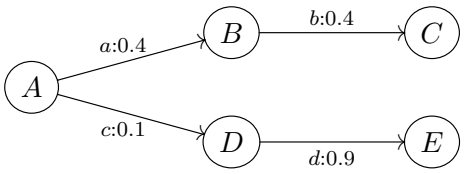
Différentes propriétés peuvent être désirables pour les opérateurs d'agrégation : une invariance à la longueur de la liste des scores (implicitement du chemin), une pénalisation en fonction du temps, des possibilités de compensation, etc.

Opérateur de moyenne géométrique Soit $\epsilon > 0$ qui permet de ne pas avoir à restreindre l'évaluation à des valeurs strictement supérieures à 0.

$$
agg_{geo}(s_0, s_1, \ldots, s_{n-1}) = \sqrt[n]{\prod_{i=0}^{n-1} \max(\epsilon, s_i)} \tag{11}
$$

Cet opérateur permet une plus grande expressivité que la moyenne arithmétique. Il est plus sensible à une variance importante. Même s'il reste un opérateur de compensations, pour "rattraper" un score faible, la compensation doit être plus forte qu'avec une moyenne arithmétique. Par exemple :

- $1 - agg_{geo}([0.5, 0.5, 0.5]) = 0.5$
- $1 - agg_{geo}([0.5, 0.3, 0.7]) = 0.53$ la moyenne arithmétique est bien de 0.5 mais, l'opérateur pénalise le faible score de 0.3
- $1 - agg_{geo}([0.5, 0.3, 0.8]) = 0.5$ la moyenne arithmétique est de 0.53.



(a) LTS

chemin	1 - moyenne	1 - agg_{geo}
ABC	0.6	**0.6**
ADE	**0.5**	0.7

(b) Coûts

FIGURE 4 – Exemple opérateur géométrique

On observe sur la figure 4 que le coût associé au chemin [ABC] est plus faible que celui du chemin [ADE] même si ce dernier possède une moyenne arithmétique plus élevée. En effet, la variance plus importante de [ADE] est pénalisée en utilisant l'opérateur géométrique. On peut imaginer des situations simples où l'on préfère faire une action neutre comme mettre du sirop dans un verre d'eau plutôt que de mettre du poison puis un antidote avant de le servir (en utilisant une norme relative au bien et au mal).

Opérateur gamma (temporel) Soit $\gamma \in]0,1]$, $\Gamma = \frac{1-\gamma^N}{1-\gamma}$ ($\Gamma = N$ si $\gamma = 1$)

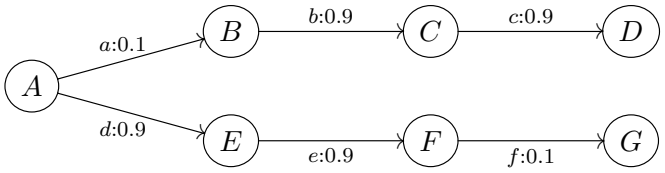
$$agg_\gamma(s_0, s_1, \ldots, s_{n-1}) = \frac{\sum_{i=0}^{N-1} \gamma^i s_i}{\Gamma} \tag{12}$$

Cet opérateur permet d'accorder une grande importance aux événements au début d'une trace puis de manière exponentiellement décroissante à ceux qui arrivent à un temps avancé. On peut facilement définir de manière symétrique l'agrégateur $agg_{\gamma\ late}$:

$$agg_{\gamma\ late}(s_0, s_1, \ldots, s_{n-1}) = \frac{\sum_{i=0}^{N-1} \gamma^{N-1-i} s_i}{\Gamma} \tag{13}$$

Ce dernier accorde plus d'importance aux scores en fin de traces qu'à ceux du début (voir Figure 5). La valeur du γ possède un rôle essentiel pour cet opérateur et doit être fixée relativement à la longueur de la liste plus la liste des scores est longue plus γ doit être proche de 1 et inversement pour des courtes listes, il est préférable de fixer γ à des valeurs proches de 0.

On peut utiliser l'opérateur gamma si l'on souhaite pénaliser des événements avec un faible score en début de trace. Par exemple, si on utilise une norme statistique, on souhaite exclure les chemins qui dès les premiers temps possèdent un événement statistiquement peu probable même si après tous les événements qui en découlent ont une forte possibilité de se produire.



(a) LTS

chemin	1 - moyenne	1 - agg_γ	1- $agg_{\gamma late}$
ABCD	**0.3**	0.42	**0.21**
AEFG	**0.3**	**0.21**	0.42

(b) Coûts ($\gamma = 0.7$)

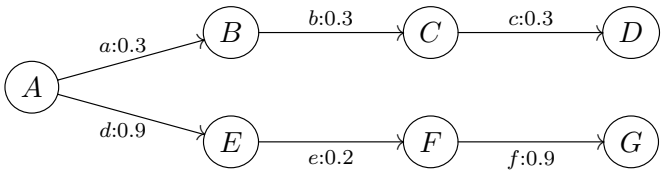
FIGURE 5 – Exemple opérateurs gamma

Les opérateurs gamma permettent de prendre en compte le temps dans l'agrégation des scores, ils ne sont pas invariants à la longueur de la trace et ne permettent pas une réelle compensation entre des scores si ils sont éloignés temporellement.

Opérateur seuil Soit $\lambda \in [0,1]$:

$$agg_{seuil}(s_0, \ldots, s_{n-1}) = \begin{cases} 0 & si \quad \exists i < n \ tel\ que\ s_i < \lambda \\ 1 & sinon \end{cases} \tag{14}$$

L'opérateur de seuil permet soit de pénaliser complètement un contrefactuel si un événement possède un score inférieur au seuil fixé soit de ne pas pénaliser si un tel événement n'apparaît pas dans la trace voir Figure 6.



(a) LTS

chemin	1 - moyenne	1 - agg_{seuil}
ABCD	0.6	**0**
AEFG	**0.33**	1

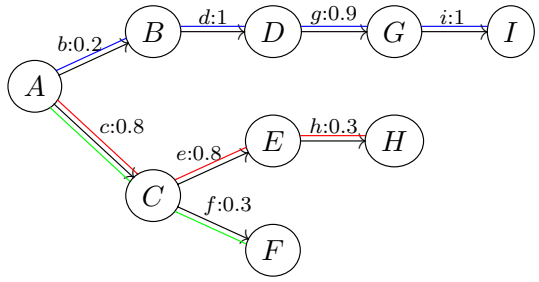
(b) Coûts ($\lambda = 0.25$)

FIGURE 6 – Exemple opérateurs de seuil

Cet opérateur peut présenter un intérêt dans des cas critiques où l'on se refuse de passer sous un seuil de score de normativité quel que soit le reste des événements.

3.3.5 Sélection des événements à évaluer

On peut vouloir agréger l'ensemble des scores d'un chemin causal (comme fait jusqu'alors dans les exemples). Mais comme le travail porte in fine sur une évaluation de contrefactuel, on peut vouloir se concentrer uniquement sur les transitions auxquelles une alternative existe, c'est-à-dire les états avec strictement plus d'une transition sortante.



(a) LTS

chemin	1 - agg_{geo}	événements comptés
bleu	0.**35**	b, d, g, i
rouge	0.42	c, e, h
vert	0.51	c, f

(b) Coûts globaux ($\gamma = 0.7$)

chemin	1 - agg_{geo}	événements comptés
bleu	0.8	b
rouge	**0.2**	c, e
vert	0.51	c, f

(c) Coûts sur les choix ($\gamma = 0.7$)

FIGURE 7 – Évaluation sur les choix

Comme on l'observe sur la Figure 7, considérer seulement les événements avec une alternative modifie complètement les résultats, on évalue la qualité des choix pris durant un scénario et non plus le scénario dans sa globalité. Lorsque l'on évalue la normativité totale, c'est le chemin bleu qui minimise le coût, hors ce scénario pour l'unique choix possible qu'il a, a effectué le pire choix. Contrairement à ce dernier, le chemin rouge sur les deux événements qui présentent une alternative prend les événements qui maximise le score. En réduisant les événements à considérer c'est le chemin rouge qui minimise le coût.

3.4 Critère de validité

Le critère permet de filtrer les contrefactuels non valides. Un contexte causal contrefactuel et considéré comme valide par rapport au contexte causal observé si l'état terminal de la trace du contrefactuel est différent de celui de l'observation et que son coût est inférieur à un seuil fixé.

Soit $\chi = (\pi, S_0, \tau)$ le contexte causal observé avec $\sigma^{\chi}_{|state} = [S^{\chi}(0), \ldots, S^{\chi}(n)]$ et $\chi' = (\pi', S'_0, \tau)$ le contexte causal contrefactuel que l'on évalue, avec $\sigma^{\chi'}_{|state} = [S^{\chi'}(0), \ldots, S^{\chi'}(m)]$ et $\delta \in \mathbb{R}^+$, χ' est valide si :

$$S^{\chi}(n) \neq S^{\chi'}(m) \quad \text{et} \quad Coût_{\tau}(\chi, \chi') < \delta. \tag{15}$$

On peut aussi envisager d'affiner le critère de validité en rajoutant des seuils propres à v et d_obs.

4 Transition d'un framework RAC vers LTS : PDDL

Les LTS ne constitue pas directement un langage pour le domaine du Raisonnement sur l'Action et le changement (RAC) mais, comme dit dans Sarmiento 2025 leur utilisation permet d'unifier des raisonnements sur différents frameworks ainsi que de lever certaines ambiguïtés. Tout formalisme pouvant être représenté sous forme de LTS bénéficie ainsi des réflexions et observation de ce travail. Pour illustrer la possibilité de transition, nous allons développer sur la traduction d'un problème PDDL vers LTS. La représentation LTS permet de travailler sur la comparaison de contexte causal contrefactuel, mais on laisse le travail de recherche et de résolution de problème de planification aux outils préexistants. En effet, la complexité exponentielle d'un LTS en fonction du nombre d'états et d'action n'est pas adapté à la recherche de politique.

4.1 Rappel PDDL

Un problème de planification est décrit en PDDL par deux types de fichiers. Le premier définit le domaine de planification, il contient la déclaration des types, des actions (effets et préconditions) ainsi que les axiomes et les règles du domaine. Le second fichier définit le problème, il indique les objets, les constantes spécifiques ainsi que la situation de départ et l'objectif. Un exemple se trouve en annexe 4.4

4.2 Construction d'un LTS à partir d'un problème PDDL

Pour construire un LTS à partir d'un couple de fichier domaine/problème, on commence par instancier toutes les actions possibles à partir de leurs définitions et des objets du problème, ce processus est explicité dans ci-dessous.

```
def instantiate(actions, objects):
    instanciated = []
```

```
    for action in actions:
        params = action.parameters
        for args in product(objects, repeat=len(params)):
            val = dict(zip(params, args))
            name, precond,effect =  action.apply(val)
            instanciated.append((name, precond, effect))
    return instanciated
```

Une fois l'ensemble de toutes les actions instanciées, avant de pouvoir construire le LTS, on définit la fonction de satisfaction de préconditions. Pour un ensemble de préconditions et un état représenté par un ensemble de fluents, elle vérifie si les préconditions sont respectées dans l'état. Cette fonction gère les préconditions négatives, disjonctives et conjonctives.

```
def statisfied(preconditions,state):
    if preconditions conjonctives:
        return all(statisfied(p,state) for p in preconditions)
    if preconditions negatives:
        return not statisfied(preconditions,state)
    if preconditions disjonctives:
        return any(statisfied(p,state) for p in preconditions)
    return precondition in state
```

La seconde fonction qui est nécessaire avant celle de construction du graphe est la fonction d'application des effets positifs et négatifs d'une action dans un état.

```
def apply_effects(effects,state):
    new_state = state
    for f in effects:
        if isinstance(f, Not):
            new_state.discard(f) # effet negatif
        else:
            new_state.add(f) #effet positif
    return new_state
```

Avec ces trois fonctions, on peut maintenant définir la fonction de construction du LTS associé au problème PDDL. Un état est défini par l'ensemble des prédicats qui sont vrais dans ce dernier. On utilise une pile pour progresser dans la construction du graphe, à chaque nouvel état, on observe parmi toutes les actions possibles lesquelles sont applicables en fonction de leurs préconditions. Si une action est applicable, on ajoute une nouvelle arête qui a pour état d'arrivée soit un état préexistant soit un nouvel état que l'on ajoute à la pile. Les états finaux sont les états dans lesquelles les conditions de l'objectif sont respectés. Pour les distinguer des autres états une arête réflexive est ajoutée.

```
def build_graph(actions, init_state, goals):
```

```
state = init_state
G = graph
G.add_node(state)
queue = [state]
visited = {}
while queue:
    current_node = queue.pop(0)
    if current_node in visited:
        continue
    visited.add(current_node)

    for preconditions, effects in actions:
        if satisfied(preconditions, current_node):
            new_node = apply_effect(effects, current_node)

        exist = new_node in G.nodes
        if not exist:
            G.add_node(new_node)
        else:
            continue

        G.add_edge(current_node, new_node)

        if new_node not in visited:
            queue.append(visited)

for node in G.nodes:
    if satisfied(goals, node):
        G.add_edge(node,node, action = '$\\varnothing')
```

4.3 Remarques et limites

Il est possible de continuer de tirer parti de PDDL et de ses solveurs pour extraire de manière automatique des politiques qui nous permettent par la suite de travailler sur des contextes causaux. Pour construire la politique, on avance dans le graphe depuis l'état de départ en attribuant à chaque état dans lequel on se trouve l'action associée dans le plan. Si besoin ces politiques partielles peuvent ensuite être complétées aléatoirement ou selon une heuristique.

Il est à noter que les LTS correspondent à une sémantique et non pas à de vrais outils de travail, leur utilité pour des applications concrètes reste limitée et ils seront surtout utilisés de manière formelle pour raisonner. Pour des applications à des exemples simples, ils permettent une meilleure visualisation, on considère donc suffisant un parseur simple comme présenté ci-dessus. L'idée de cette section étant principalement d'expliciter la faisabilité d'une transition d'un framework RAC vers des LTS, de possibles améliorations sont laissées en suspend, car ne représentant pas le cœur du travail. On peut par exemple noter que ce parseur ne gère que des problèmes PDDL contenant uniquement

le requirement STRIPS, disjunctive-preconditions et negative-preconditions.

4.4 Exemple PDDL

Nous appliquons dans cette section notre parser sur une instanciation de problème PDDL issus de `https://github.com/arii/planning_exercises/tree/master`. Le domaine correspond à un domaine Block World classique avec les quatre actions classiques de manipulation d'objet (*"pick-up"*, *"put-down"*, *"stack"*,*"unstack"*), la définition complète du domaine se trouve dans la section F.1. Le problème contient deux objets bloc A et bloc B, initialement le bras ne tient rien, le bloc B est posé sur la table et le bloc A au-dessus du b. L'objectif est de poser A sur la table et que rien ne soit posé sur B. Les conditions de l'objectif sont donc respecté dans deux états : A est posé sur la table et B est posé sur la table, ou bien A est posé sur la table et B est posé sur A.

Définition du domaine

Définition du problème

```
(define (problem bw-simple)
  (:domain bw)
  (:objects a b )
  (:init (clear a) (arm-empty) (on a b) (on-table b))
  (:goal (and (on-table a) (clear b))))
```

On obtient pour l'exemple 4.4 le graphe suivant :

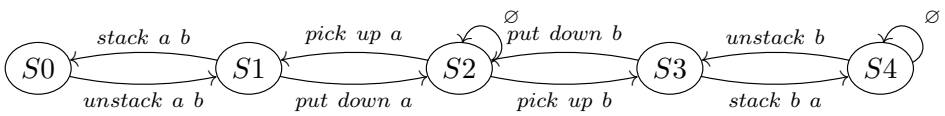


FIGURE 8 – LTS obtenu pour le problème simple de block world. Avec S0 ={'clear a', 'on a b', 'on table b', 'arm-empty'}, S1 ={'on table b', 'clear b', 'holding a'}, S2 ={'on table b','arm empty", 'clear b','clear a', 'on table a'}, S3 ={'holding b','on table a', 'clear a'}, S4 ={'on b a', 'on table a', 'arm empty', 'clear b'}

5 Application : exemple du poison

Dans cette partie, nous allons présenter un exemple concret d'application du travail présenté précédemment. Cette section est organisée sous forme de pipeline, nous partons d'un problème défini en PDDL que nous convertissons en LTS, nous ajoutons une évaluation experte avant d'appliquer différentes configurations de la fonction de coût pour en analyser les nuances. L'exemple défini correspond à une situation dans laquelle se trouve un verre qui contient à l'origine de l'eau. On considère que différents événements peuvent se produire pour altérer la potabilité de ce dernier et

entraîner ou non la mort de la personne qui le boit. Les événements sont évalués selon une norme que l'on laisse floue, mais qui correspond approximativement à une norme du bien dans le sens où un événement est jugé comme peu normal s'il est associé à une nuisance (comme empoisonner le verre qui peut entraîner la mort) et comme très normal s'il entraîne un bénéfice (verser un antidote dans un verre empoisonné permet de sauver une personne). La modélisation de la situation et de l'évaluation est entièrement subjective et pourrait être complexifié, mais nous souhaitons garder un exemple simple pour plus de lisibilité.

5.1 De PDDL vers un LTS

La définition des fichiers PDDL se trouve en annexe F.2. Ci-dessous le LTS résultant de l'application du parseur.

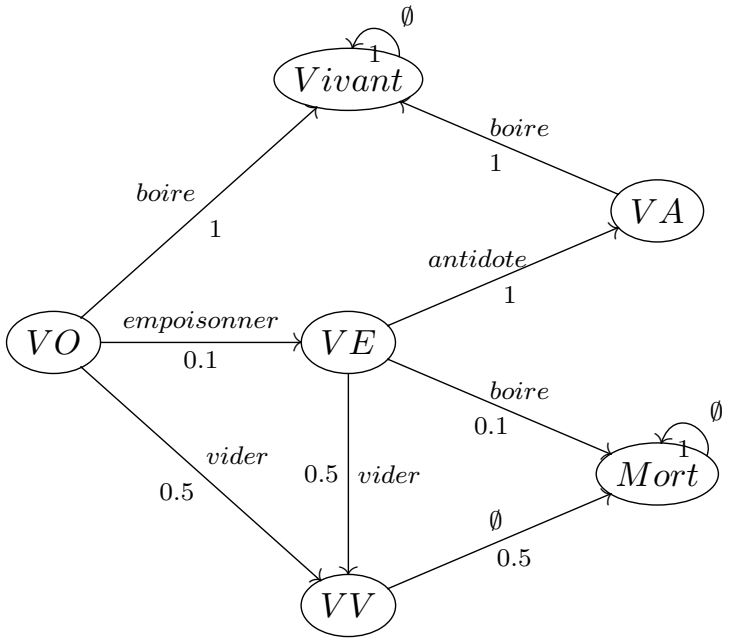
$$\mathbb{F} = \{VerreOK, VerreEmpoisonné, VerreVide, VerreAntidote, Mort, Vivant\}$$
$$\mathbb{E} = \{empoisonner, antidote, boire_{VO}, boire_{VA}, boire_{VE}, vider_{VO}, vider_{VE}\}$$
$$\mathbb{A} = \{Vivant, Mort\}$$
$$\mathbb{T} = \mathbb{N}$$
$$pre(boire_{VO}) = VO \quad pre(antidote) = VE$$
$$pre(boire_{VA}) = VA \quad pre(boire_{VE}) = VE$$
$$pre(empoisonner) = VO \quad pre(vider_{VO}) = VO$$
$$pre(vider_{VE}) = VE$$
$$pre(\emptyset) = VV \vee Mort \vee Vivant$$
$$\mathbb{S} = \left\{ \begin{array}{r} VO = VO \wedge \neg VE \wedge \neg VV \wedge \neg VA \wedge \neg \text{Mort} \wedge \neg \text{Vivant}, \\ VE = \neg VO \wedge VE \wedge \neg VV \wedge \neg VA \wedge \neg \text{Mort} \wedge \neg \text{Vivant}, \\ VV = \neg VO \wedge \neg VE \wedge VV \wedge \neg VA \wedge \neg \text{Mort} \wedge \neg \text{Vivant}, \\ VA = \neg VO \wedge \neg VE \wedge \neg VV \wedge VA \wedge \neg \text{Mort} \wedge \neg \text{Vivant}, \\ \text{Mort} = \neg VO \wedge \neg VE \wedge \neg VV \wedge \neg VA \wedge \text{Mort} \wedge \neg \text{Vivant}, \\ \text{Vivant} = \neg VO \wedge \neg VE \wedge \neg VV \wedge \neg VA \wedge \neg \text{Mort} \wedge \text{Vivant} \end{array} \right\}$$

Pour alléger la notation, on fusionne la notation de $boire_{VO}, boire_{VA}, boire_{VE}$ en $boire$ et de

$vider_{VO}, vider_{VE}$ en $vider$.

$$\tau = \left\{ \begin{array}{r} (VE, vider, VV), (VE, boire, Mort), (VE, antidote, VA), (Mort, \emptyset, Mort) \\ (VV, \emptyset, Mort), (VA, boire, Vivant), (Vivant, \emptyset, Vivant), (VO, boire, vivant) \\ (VO, boire, vivant), (VO, empoisonner, VE), (VO, vider, VV), \end{array} \right\}$$

$$expert_judge_\tau = \left\{ \begin{array}{l} VO : \{boire : 1,\ vider : 0.5,\ emp : 0.1\}, \\ VE : \{antidote : 1,\ boire : 0.1,\ vider : 0.5\}, \\ VV : \{\emptyset : 0.5\}, \\ VA : \{boire : 1\}, \\ \text{Mort} : \{\emptyset : 1\}, \\ \text{Vivant} : \{\emptyset : 1\} \end{array} \right\}$$



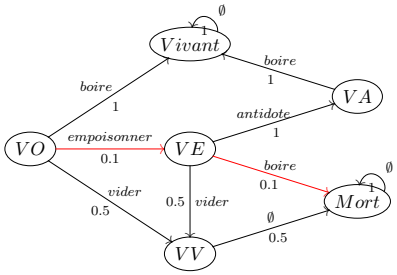
(16)

5.2 Définition des contextes causaux

On réduit le champ de travail en ne travaillant que sur des contextes causaux partageant le même état de départ S_0 et une politique correcte relativement au LTS $< \mathbb{S}, \tau >$.

- Contexte causal $\chi_1 = (\pi_1, S_0, \tau)$ avec $\pi_1 \in \Pi_\tau$:

$\pi_1(VO) = \{empoisonner\}, \quad \pi_1(VE) = \{boire\}, \quad \pi_1(mort) = \{\emptyset\}$
$\pi_1(VV) = \{\emptyset\}, \quad \pi_1(VA) = \{boire\}, \quad \pi_1(vivant) = \{\emptyset\}$



On obtient :

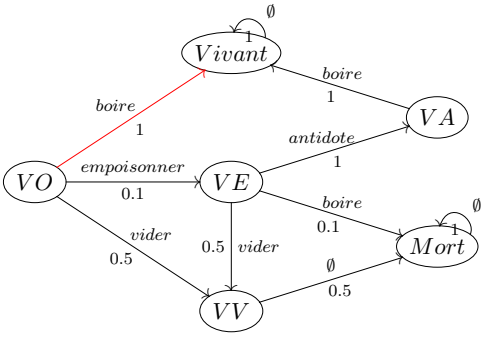
— $\sigma^{\chi_1} = [VO, empoisonner, VE, boire, Mort]$

— $\sigma^{\chi_1}_{|state} = [VO, VE, Mort]$

— $\sigma^{\chi_1}_{|event} = [empoisonner, boire]$

- Contexte causal $\chi_2 = (\pi_2, S_0, \tau)$avec $\pi_2 \in \Pi_\tau$:

$\pi_2(VO) = \{boire\}, \quad \pi_2(VE) = \{boire\}, \quad \pi_2(mort) = \{\emptyset\}$
$\pi_2(VV) = \{\emptyset\}, \quad \pi_2(VA) = \{boire\}, \quad \pi_2(vivant) = \{\emptyset\}$



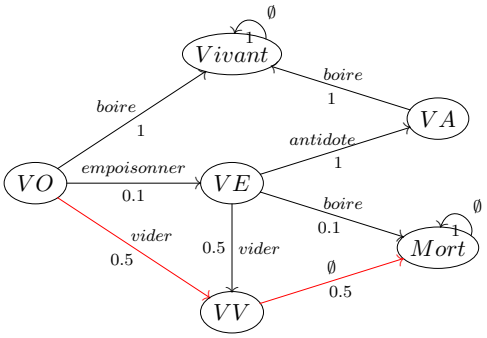
— $\sigma^{\chi_2} = [VO, boire, Vivant]$

— $\sigma^{\chi_2}_{|state} = [VO, Vivant]$

— $\sigma^{\chi_2}_{|event} = [boire]$

- Contexte causal $\chi_3 = (\pi_3, S_0, \tau)$ avec $\pi_3 \in \Pi_\tau$:

$\pi_3(VO) = \{vider\}, \quad \pi_3(VE) = \{boire\}, \quad \pi_3(mort) = \{\emptyset\}$
$\pi_3(VV) = \{\emptyset\}, \quad \pi_3(VA) = \{boire\}, \quad \pi_3(vivant) = \{\emptyset\}$



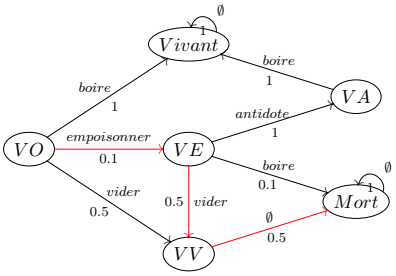
— $\sigma^{\chi_3} = [VO, vider, VV, \emptyset, Mort]$

— $\sigma^{\chi_3}_{|state} = [VO, VV, Mort]$

— $\sigma^{\chi_3}_{|event} = [vider, \emptyset]$

- Contexte causal $\chi_4 = (\pi_4, S_0, \tau)$ avec $\pi_4 \in \Pi_\tau$:

$\pi_4(VO) = \{empoisonner\}, \quad \pi_4(VE) = \{vider\}, \quad \pi_4(mort) = \{\emptyset\}$
$\pi_4(VV) = \{\emptyset\}, \quad \pi_4(VA) = \{boire\}, \quad \pi_4(vivant) = \{\emptyset\}$



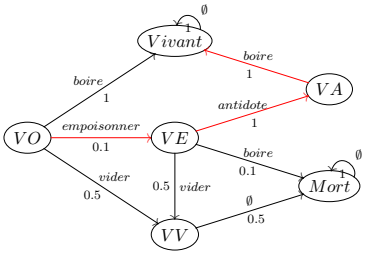
— $\sigma^{\chi_4} = [VO, empoisonner, VE, vider, VV, \emptyset, Mort]$

— $\sigma^{\chi_4}_{|state} = [VO, VE, VV, Mort]$

— $\sigma^{\chi_4}_{|event} = [empoisonner, vider, \emptyset]$

- Contexte causal $\chi_5 = (\pi_5, S_0, \tau)$ avec $\pi_5 \in \Pi_\tau$:

$$\pi_5(VO) = \{empoisonner\}, \quad \pi_5(VE) = \{antidote\}, \quad \pi_5(mort) = \{\emptyset\}$$
$$\pi_5(VV) = \{\emptyset\}, \quad \pi_5(VA) = \{boire\}, \quad \pi_5(vivant) = \{\emptyset\}$$



- $\sigma^{\chi_5} = [VO, empoisonner, VE, antidote, VA, boire, Vivant]$
- $\sigma^{\chi_5}_{|state} = [VO, VE, VA, Vivant]$
- $\sigma^{\chi_5}_{|event} = [empoisonner, antidote, boire]$

5.3 Définition des politiques optimales

Politique optimale sans planification :

$$\pi_\tau^{opt1}(VO) = \{boire\}, \quad \pi_\tau^{opt1}(VE) = \{antidote\}, \quad \pi_\tau^{opt1}(VV) = \{\emptyset\}, \quad \pi_\tau^{opt1}(Mort) = \{\emptyset\}$$
$$\pi_\tau^{opt1}(VA) = \{boire\}, \quad \pi_\tau^{opt1}(Vivant) = \{\emptyset\}$$

Politique optimale avec planification :

$$\pi_\tau^{opt2}(VO) = \{boire\}, \quad \pi_\tau^{opt2}(VE) = \{antidote\}, \quad \pi_\tau^{opt2}(VV) = \{\emptyset\}, \quad \pi_\tau^{opt2}(Mort) = \{\emptyset\} \quad (17)$$
$$\pi_\tau^{opt2}(VA) = \{boire\}, \quad \pi_\tau^{opt2}(Vivant) = \{\emptyset\} \quad (18)$$

On note $\forall S \in \mathbb{S}, \quad \pi_\tau^{opt1}(S) = \pi_\tau^{opt2}(S)$. On notera par la suite simplement π_τ^{opt}.

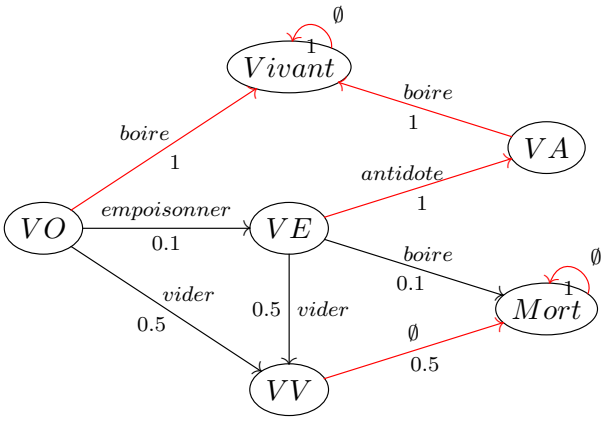


FIGURE 9 – Politique optimale : les flèches rouges indiquent dans chaque état l'événement choisi par la politique optimale

5.4 Expérimentations

En fonction de la configuration choisie (extraction d'une politique optimale, exploitation directe, évaluation de la normalité globale, sur les événements avec alternatives uniquement) et des hyper paramètres ($\lambda_1, \lambda_2, \gamma$...) on obtient différents résultats. Une connaissance à priori des contraintes que l'on souhaite appliquer lors de l'évaluation permet de guider le classement.

Dans les exemples suivants, on considère, si ce n'est pas précisé, que le contexte causal observé est χ_1, on fixe $\lambda_1 = \lambda_2 = 1$ et un coût maximal $\delta = 2$ (avec cette valeur la validité ne dépends que de la différence entre les deux états finaux des traces de l'observation et du contrefactuel).

Priorité à des contrefactuels avec des scores d'événements consistants : On choisit pour la fonction de vraisemblance la formule 3.3.4 avec comme opérateur d'agrégation agg_{geo} et on agrège les scores de tous les événements des scénarios. On souhaite dans cette configuration pénaliser des traces avec une variance de score importante.

On obtient les résultats suivants :

CF	classement	d_{obs}	v	$d_{\text{obs}} + v$	trace	valide
1	χ_2	0.67	0.0	0.67	$[VO, boire, Vivant]$	✓
3	χ_3	0.33	0.5	0.83	$[VO, vider, VV, \emptyset, Mort]$	×
4	χ_4	0.25	0.71	0.96	$[VO, empoisonner, VE, vider, VV, \emptyset, Mort]$	×
2	χ_5	0.50	0.54	1.04	$[VO, empoisonner, VE, antidote, VA, boire, Vivant]$	✓

TABLE 1 – Résultats pour la priorité à la normativité

On observe dans la table 1 que le contexte contrefactuel avec le coût minimal est le contexte $\chi 2$ sa distance à l'observation est de 0.67 et sa vraisemblance de 0 (l'unique événement *boire* a un score de 1). Ce contrefactuel est suivi du contexte χ_5, plus proche de l'observation (coût de 0.5) mais le scénario associé contient l'événement *empoisonner* qui fait augmenter le coût. Les deux premiers contrefactuels χ_2 et χ_5 sont valides.

Interdiction de dépasser un seuil On utilise l'opérateur d'agrégation avec un seuil fixé à $\lambda = 0.2$ pour pénaliser les contrefactuels dont le scénario contient un empoisonnement. On fixe aussi un coût maximal de 0.5 pour le coût de vraisemblance.

CF	classement	d_{obs}	v	$d_{\text{obs}} + v$	trace	valide
2	χ_2	0.67	0.0	0.67	$[VO, boire, Vivant]$	✓
1	χ_3	0.33	0.0	0.33	$[VO, vider, VV, \emptyset, Mort]$	×
3	χ_4	0.25	1.0	1.25	$[VO, empoisonner, VE, vider, VV, \emptyset, Mort]$	×
4	χ_5	0.50	1.0	1.50	$[VO, empoisonner, VE, antidote, VA, boire, Vivant]$	×

TABLE 2 – Résultats pour la priorité à la normativité

Dans cette configuration uniquement le contexte causal χ_2 est valide. En rajoutant une contrainte

sur le coût de vraisemblance, on exclut automatiquement les contrefactuels qui possèdent un événement inférieur au seuil, c'est ce qu'on observe pour χ_5.

Normativité en début de scénario On utilise maintenant l'opérateur agg_γ, avec $\gamma = 0.25$, qui avantage les scénarios qui ont des scores élevés sur les événements en début de trace en considérant peu ceux de la fin de la trace.

CF	classement	d_{obs}	v	$d_{\text{obs}} + v$	trace	valide
1	χ_2	0.67	0.0	0.67	$[VO, boire, Vivant]$	✓
2	χ_3	0.33	0.5	0.83	$[VO, vider, VV, \emptyset, Mort]$	×
3	χ_4	0.25	0.80	1.05	$[VO, empoisonner, VE, vider, VV, \emptyset, Mort]$	×
4	χ_5	0.50	0.69	1.19	$[VO, empoisonner, VE, antidote, VA, boire, Vivant]$	✓

TABLE 3 – Priorité à la normativité en début de scénario

On observe avec la table 3 que le contrefactuel avec le plus faible coût correspond à χ_2 qui a le premier événement avec un score le plus élevé (*boire* =1) suivit de χ_3 qui a un score pour son premier événement de 0.5. Les deux derniers contextes causaux possède en premier événement *empoisonner* ce qui vient fortement augmenter le coût.

Normativité en fin de scénario On utilise l'opérateur $gamma_{late}$ avec $\gamma = 0.25$. On priorise la normativité en fin de trace, couplé au coût sur la distance à l'observation, on cherche des contrefactuels qui ne diffèrent de l'observation qu'en fin de trace, mais qui reste le plus proche possible au début.

CF	classement	d_{obs}	v	$d_{\text{obs}} + v$	trace	valide
2	χ_2	0.67	0.0	0.67	$[VO, boire, Vivant]$	✓
4	χ_3	0.33	0.5	0.83	$[VO, vider, VV, \emptyset, Mort]$	×
3	χ_4	0.25	0.52	0.77	$[VO, empoisonner, VE, vider, VV, \emptyset, Mort]$	×
1	χ_5	0.50	0.04	0.54	$[VO, empoisonner, VE, antidote, VA, boire, Vivant]$	✓

TABLE 4 – Priorité à la normativité en fin de scénario

La table des résultats 4 montre que dans cette configuration, c'est χ_5 qui a le coût le plus faible, malgré l'empoisonnement, on s'éloigne de l'observation le plus tard possible.

Normativité du contrefactuel avec distance à une politique optimale On choisit pour cette configuration de calculer la distance à la politique optimale définie par 17. Le choix est fait de ne considérer que les événements pour lesquelles une alternative existe.

Les résultats présents dans la table 5 indiquent qu'avec cette configuration, le contexte causal avec le plus faible coût est le contexte χ_2 suivi de χ_5 les deux contrefactuels valides. Concernant la fonction de vraisemblance : dans la trace associée à χ_2 un seul événement est présent et un choix est possible, le choix fait est l'événement *boire* qui correspond à l'événement choisi par la politique

CF	classement	d_{obs}	v	$d_{\text{obs}} + v$	trace	valide
1	χ_2	0.67	0.0	0.67	$[VO, boire, Vivant]$	✓
4	χ_3	0.33	1.0	1.33	$[VO, vider, VV, \emptyset, Mort]$	×
3	χ_4	0.25	1.0	1.25	$[VO, empoisonner, VE, vider, VV, \emptyset, Mort]$	×
2	χ_5	0.50	0.5	1.0	$[VO, empoisonner, VE, antidote, VA, boire, Vivant]$	✓

TABLE 5 – Résultats distance à la politique optimale.

optimale donc le coût associé est de 0. Pour χ_5, deux événements avec alternative sont présents dans la trace *empoisonner* qui n'est pas l'événement indiqué par la politique optimale, et *antidote* qui est similaire à la politique optimale. Sur les deux événements un seul est égal à la politique optimale donc un coût de 0.5 est appliqué. Pour χ_3 et χ_4 tous les choix possibles différents de la politique optimale un coût d'un leur est attribué.

A Exemple contexte causal non valide pour la fonction de coût

Soit, le LTS $\langle \mathbb{S}, \tau \rangle$ avec :

$$\mathbb{S} = \{S_0, S_1, S_2, S_3\}$$
$$\tau = \{(S_0, a, S_1), (S_0, b, S_2), (S_2, c, S_3), (S_2, d, S_4), (S_1, \emptyset, S_1), (S_3, \emptyset, S_3), (S_4, \emptyset, S_4)\}$$

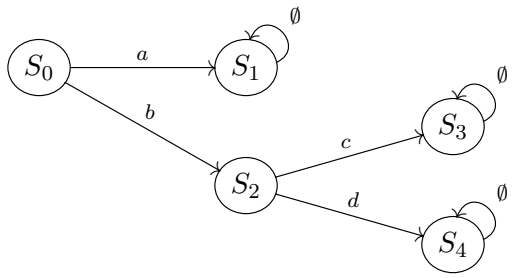


FIGURE 10 – Exemple de LTS

On définit : $\chi_1 = (\pi_1, S_0, \tau)$, (π_2, S_3, τ) (π_3, S_0, τ) et avec :

$$\pi_1(S_0) = a \quad \pi_1(S_2) = c \quad et \quad \forall i \in [1, 3, 4], \pi_1(S_i) = \emptyset$$
$$\pi_2(S_0) = a \quad \pi_2(S_2) = d \quad et \quad \forall i \in [1, 3, 4], \pi_2(S_i) = \emptyset$$
$$\pi_3(S_0) = e \quad \pi_3(S_2) = d \quad et \quad \forall i \in [1, 3, 4], \pi_3(S_i) = \emptyset$$

A.1 Fonction *divergence*

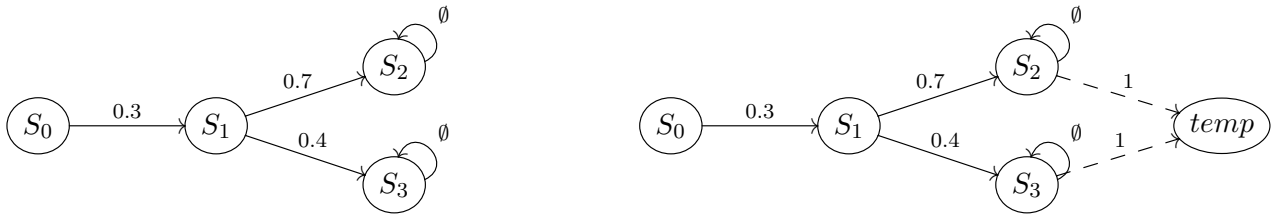
$divergence(\chi_1, \chi_2) = +\infty$ même si ces deux contextes causaux sont différents le coût est de $+\infty$, les politiques divergent uniquement en S_2 hors cet état est absent de $\sigma^{\chi_1} = [S_0, S_1]$. Les politiques sont égales sur la trace.

A.2 Fonction *correct*

$correct(\chi_3) = +\infty$ la politique π_3 n'étant pas correcte relativement au LTS. $\nexists S'$ *tel que* $(S_0, e, S') \in \tau$

B Construction d'une politique optimale avec planification et l'utilisation de l'algorithme A*

Les algorithmes de plus court chemin nécessitent l'indication d'un état de départ et d'arrivée. Dans notre cas, on souhaite trouver le chemin qui minimise le coût sans contraindre à une destination



(a) LTS original

(b) LTS augmenté

FIGURE 11 – Construction du graphe pour l'application d'un algorithme de plus court chemin

fixe. Pour permettre cela, on crée un LTS temporaire, à partir de notre LTS de travail, auquel on ajoute un état "poubelle" (voir Figure 11). On relie chaque état terminal du LTS original à ce nouvel état, et chaque transition entre un état terminal et l'état possède un score égal (par exemple 1). L'ajout de cet état nous permet d'appliquer un algorithme de recherche du plus court chemin en fixant l'état d'arrivée à l'état ajouté.

C Vérification des propriétés pour les opérateurs

Rappel : Définition et propriétés d'un opérateur d'agrégation : $Agg : \bigcup_{n\in\mathbb{N}} [0,1]^n \to [0,1]$

- identité si $n = 1$: $Agg(u) = u$
- conditions aux limites : $Agg(0,\dots,0) = 0$ et $Agg(1,\dots,1) = 1$
- monotonie croissante : $Agg(u_1,\dots,u,\dots,u_n) \leq Agg(u_1,\dots,v,\dots,u_n)$ si $u \leq v$

C.1 Opérateur géométrique

Soit $\epsilon > 0$

$$agg_{geo}(s_0, s_1, \dots, s_{n-1}) = \exp\left(\frac{1}{N}\sum_{i<n} \log(\max(s_i, \epsilon))\right) \tag{19}$$

Identité

$$agg_{geo}(u) = \exp(\log(\max(u, \epsilon))) = \max(u, \epsilon)$$

Si $u < \epsilon$, $agg_{geo}(u) = \epsilon \neq u$. La propriété d'identité n'est pas respectée.

Conditions aux limites

$$agg_{geo}(0,\dots,0) = \exp\left(\frac{1}{N}\sum_{i<n} \log(\epsilon)\right) = \epsilon \neq 0$$

$$agg_{geo}(1,\dots,1) = \exp\left(\frac{1}{N}\sum_{i<n} \log(1)\right) = 1$$

Pour la limite en zéro la condition n'est pas respectée.

Monotonie croissante Soit $u \leq v$, $Agg(u_1, \ldots, u, \ldots, u_n)$ ne diffère de $Agg(u_1, \ldots, v, \ldots, u_n)$ que par un seul terme de la somme. On veut montrer que $\log(\max(u, \epsilon)) \leq \log(\max(v, \epsilon))$, hors $\max(u, \epsilon) \leq \max(v, \epsilon)$ et donc par croissance du log, on obtient : $\log(\max(u, \epsilon)) \leq \log(\max(v, \epsilon))$. Donc la somme est plus petite et l'exponentielle aussi. La propriété de monotonie croissante est respectée.

C.2 Opérateur gamma

Soit $\gamma \in]0, 1]$, $\Gamma = \frac{1-\gamma^N}{1-\gamma}$ ($\Gamma = N$ si $\gamma = 1$)

$$agg_\gamma(s_0, s_1, \ldots, s_{n-1}) = \frac{\sum_{i=0}^{N-1} \gamma^i s_i}{\Gamma} \tag{20}$$

Identité Soit $u \in [0, 1]$

$$agg_\gamma(u) = \frac{\gamma^0 u}{\gamma^0} = u$$

La propriété d'identité est respectée.

Conditions aux limites

$$agg_\gamma(0, \ldots, 0) = \frac{\sum_{i=0}^{N-1} \gamma^i 0}{\Gamma} = 0$$

$$agg_\gamma(1, \ldots, 1) = \frac{\sum_{i=0}^{N-1} \gamma^i 1}{\Gamma} = 1$$

Les conditions aux limites sont respectées.

Monotonie croissante Soit $u, v \in [0, 1]$ tel que $u \leq v$

$$agg_\gamma(u_0, \ldots, u, \ldots, s_{n-1}) \leq agg_\gamma(u_0, \ldots, v, \ldots, s_{n-1}),$$

car $\gamma^i u \leq \gamma^i v$ La propriété de monotonie croissante est respectée.

C.3 Opérateur de seuil

Soit $\lambda \in]0, 1]$

$$agg_{seuil}(s_0, \ldots, s_{n-1}) = \begin{cases} 0 & si \quad \exists i < n \ tel \ que \ s_i < \lambda \\ 1 & sinon \end{cases} \tag{21}$$

Identité Soit $u \in [0,1]$

$$agg_{seuil} = \begin{cases} 0 & si \quad \exists i < n \ tel \ que \ u < \lambda \\ 1 & sinon \end{cases}$$

Si $u \neq 0$ ou $u \neq 1$ la propriété d'identité n'est pas respectée.

Conditions aux limites

$$agg_{seuil}(0, \ldots, 0) = 0 \quad agg_{seuil}(1, \ldots, 1) = 1$$

Les conditions aux limites sont respectées

Monotonie croissante Soit $u \leq v$, on veut montrer que :

$$agg_{seuil}(u_0, \ldots, u, \ldots, u_{n-1}) \leq agg_{seuil}(u_0, \ldots, v, \ldots, u_{n-1}).$$

- Si $agg_{seuil}(u_0, \ldots, v, \ldots, u_{n-1}) = 1$ on a :

$$agg_{seuil}(u_0, \ldots, u, \ldots, u_{n-1}) \leq agg_{seuil}(u_0, \ldots, v, \ldots, u_{n-1})$$

- Si $agg_{seuil}(u_0, \ldots, v, \ldots, u_{n-1}) = 0$ alors, il existe $i \in [0, n-1]$ tel que $u_i < \lambda$:
  - Si i est différents de l'indice de position de u et v, alors ce même u_i amène aussi à :

$$agg_{seuil}(u_0, \ldots, u, \ldots, u_{n-1}) = 0$$

  - Si i corresponds à l'indice de position de u et v on a $u \leq v < \lambda$ et donc :

$$agg_{seuil}(u_0, \ldots, u, \ldots, u_{n-1}) = 0.$$

La propriété de monotonie croissante est respectée.

C.4 Synthèse résultats

Opérateur	Identité	Conditions aux limites	Monotonie croissante
agg_{geo}	×	× (en 0) / ✓ (en 1)	✓
agg_{γ}	✓	✓	✓
agg_{seuil}	×	✓	✓

TABLE 6 – Synthèse des propriétés des opérateurs d'agrégation

D Digression sur le concept de sélection causale dans le domaine des sciences cognitives

De nombreuses recherches en sciences cognitives ont été menées pour tenter de modéliser la façon dont nous concevons et préférons certains scénarios contrefactuels par rapport à d'autres. Ces mécanismes sont influencés par différents critères tels que la structure causale, le temps, les normes, la valeur de l'effet produit... Même si certaines études portent, plus ou moins implicitement, sur la notion de responsabilité causale plus que sur le concept de sélection causale, des définitions et concepts peuvent être utiles pour définir et orienter les critères d'évaluation ou encore de construction de scénarios contrefactuels. Je présente ci-dessous une liste non exhaustive de notions trouvées dans la littérature pouvant être potentiellement intégrées à notre cadre.

D.1 Mutabilité et propensité des facteurs

Les notions de mutabilité (à quel point il est simple d'imaginer qu'un facteur ait pu être différent) et de propensité (la vraisemblance qu'un effet ait lieu sachant une cause) jouent selon Kominsky et al. (2015) et McGill et Tenbrunsel (2000) un rôle essentiel dans le raisonnement contrefactuel. Le premier article vise à mettre en évidence dans le processus d'évaluation causale d'un événement A, l'importance de la mutabilité et de la propensité d'un événement B nécessaire à la production de l'effet (structure causale conjointe). D'autre part dans Kominsky et al. (2015) le travail porte cette fois-ci sur le phénomène de *"causal superseding"* qui est décrit comme : le phénomène selon lequel la violation d'une norme par un agent diminue la causalité attribuée à un autre agent. Même si cet effet sort de notre cadre par son aspect multiagents, on trouve des résultats similaires à ceux de McGill et Tenbrunsel (2000). Ils avancent, eux aussi, que selon leur modèle, un facteur causal est vu comme plus faible lorsqu'un autre facteur est plus mutable, mais uniquement si ce facteur est très susceptible de produire l'effet.

D.2 Impact des normes sur le raisonnement contrefactuel

Dans (Icard, Kominsky et Knobe 2017) une mesure de force de causalité (correspondant à un compromis suffisance et nécessité) est présentée et permet de modéliser les effets de la normalité sur le jugement causal. Dans ce travail aucune distinction de types de normes (statistique ou prescriptive) n'est appliquée et les auteurs travaillent dans un cadre non déterministe ne faisant pas de différence entre les états et les événements. Cette mesure n'est donc pas directement applicable à notre cadre, mais les effets discutés présentent des intérêts potentiels. Une différence des effets de la normalité est faite selon la structure causale. Dans les cas conjonctifs, on observe un effet d'*abnormal inflation* décrit par le fait qu'une cause soit considérée comme plus causale si elle est anormale par rapport au cas où elle est normale. Le second effet pour les structures conjonctives est l'effet de *supersession* : un facteur est considéré comme plus causal lorsque les autres facteurs sont normaux. Dans le cas disjonctif, on observe l'effet d'*abnormal deflation* : une cause est considérée comme plus causale

lorsque qu'elle est normale que lorsqu'elle ne l'est pas. Et enfin toujours dans le cas disjonctif l'effet de *no supersession with disjunction* qui avance qu'une cause est tout aussi encline à être jugée comme causale lorsqu'une seconde cause est normale, que lorsqu'elle est anormale.

D.3 Impact du temps sur le raisonnement contrefactuel

Dans Henne et al. (2021), le travail porte sur l'influence qu'à la position temporelle d'un événement dans une séquence causale sur le jugement de sa responsabilité, et comment le raisonnement contrefactuel peut expliquer ces effets. Le premier effet discuté est l'effet de récence (dans un match, on juge plutôt le dernier but comme la cause de la victoire plutôt que le dernier). Mais les auteurs observent que dans des cas de surdétermination et plus particulièrement de préemption tardive, cet effet s'inverse : si chaque effet est suffisant alors la causalité est attribuée à l'événement le plus ancien. Ils étendent le *necessity-sufficiency model* présenté par Icard, Kominsky et Knobe (2017), qui avance que plus un événement fait une différence dans une simulation contrefactuelle plus, il est vu comme causal, en introduisant l'hypothèse selon laquelle les gens sont plus enclins à imaginer des contrefactuelles autour de l'événement le plus récent (quelle que soit la structure). Avec cette extension du modèle causal, le mécanisme est toujours le même (simuler le contrefactuel de l'événement le plus récent) mais c'est la structure causale qui détermine ce que la pensée contrefactuelle révèle. Dans un cas, elle indique que l'événement causal est le plus récent et dans l'autre c'est le plus ancien.

E Démonstrations des propriétés du parser

On souhaite démontrer les propriétés de correction et de terminaison du parser, on introduit pour cela les notations suivantes :

Soit $\mathcal{P} = (\mathcal{O}, \mathcal{A}, Pred, S_{init}, \mathcal{G})$ un problème PDDL avec :

- $\mathcal{O}$: l'ensemble des objets du problème
- $Pred$: l'ensemble des prédicats déclarés dans le domaine de $\mathcal{P}$
- $\mathbb{F}(\mathcal{P}) = \{p(o_1, \ldots, o_k) \mid p \in Pred, (o_1, \ldots, o_k) \in \mathcal{O}^k\}$: ensemble des prédicats instanciables du domaine (ensemble fini, car $Pred$ et $\mathcal{O}$ sont finis)
- $\mathcal{A}$: l'ensemble des actions du problème. Pour tout $a \in \mathcal{A}$ la définition du tuple :

$$(param(a), pre(a), eff^+(a), eff^-(a))$$

  et $pre(a), eff^+(a), eff^-(a) \in 2^{\mathbb{F}(\mathcal{P})}$. Avec $param(a)$ les paramètres de l'action a, $pre(a)$ ses préconditions, $eff^+(a)$ ses effets positifs et $eff^-(a)$ ses effets négatifs.
- $\mathbb{E}(\mathcal{P}) = \{a(o_1, \ldots, o_k) \mid a \in \mathcal{A}, (o_1, \ldots, o_k) \in \mathcal{O}^k, k = |param(a)|\}$: ensemble des actions instanciables à partir des objets du domaine.

- S_{init} : l'ensemble des vrais prédicats dans l'état initial (hypothèse de monde clos), $S_{init} \in 2^{\mathbb{F}(\mathcal{P})}$
- $\mathcal{G} \in 2^{\mathbb{F}(\mathcal{P})}$: formule d'état qui décrit les conditions nécessaires pour considérer qu'un état est terminal.

On introduit $\langle \mathbb{S}^*, \tau^* \rangle$ le LTS cible associé à $\mathcal{P}$ avec :

$$S_0 \text{ l'état du LTS équivalent à } S_{init} \text{ du problème } \mathcal{P}$$

$$\mathbb{S} = \{S \mid S \in 2^{\mathbb{F}(\mathcal{P})} \text{ tel qu'il existe un chemin dans le LTS entre } S_0 \text{ et } S\}$$

$$\tau^* = \{(S, \{e\}, S') \mid S \in \mathbb{S}^*, \{e\} \in 2^{\mathbb{E}(\mathcal{P})}, pre(e) \subseteq S \text{ et } S' = (S \setminus eff^-(e)) \cup eff^+(e)\}$$
$$\cup \{(S, \emptyset, S) \mid S \in \mathbb{S}^*, \mathcal{G} \subseteq S\}.$$

Et $T(\mathcal{P}) = \langle S, \tau \rangle$ avec T notre processus (parser) et donc $T(\mathcal{P})$ le LTS obtenu en l'appliquant au problème PDDL $\mathcal{P}$.

E.1 Correction

Afin de démontrer la correction de notre parser, nous allons montrer que $T(\mathcal{P}) = \langle \mathbb{S}^*, \tau^* \rangle$.
On montre dans un premier temps que $\mathbb{S} = \mathbb{S}^*$:

- $\mathbb{S} \subseteq \mathbb{S}^*$: Pour tout S appartenant $\mathcal{S}$, S a été atteint par BFS depuis S_0 en appliquant des événements dont les préconditions sont satisfaites donc il existe un chemin entre S_0 et S. On peut donc en conclure que $\mathbb{S} \subseteq \mathbb{S}^*$.
- $\mathbb{S}^* \subseteq \mathbb{S}$ Par induction avec l'hypothèse : Si $S^* \in \mathbb{S}^*$ et il existe $\{e\} \in 2^{\mathbb{E}(\mathcal{P})}$ tel que $pre(e) \subseteq S^*$ alors $S' \in \mathbb{S}^*$ avec $S' = (S\ eff^-(e)) \cup eff^+(e)$
  - Base : $S_0 \in \mathbb{S}$, car l'algorithme BFS est initialisé avec S_0
  - Induction : Soit $S \in \mathbb{S}^* \cap \mathbb{S}$ et de plus, il existe $\{e\} \in 2^{\mathbb{E}(\mathcal{P})}$ tel que $pre(e) \subseteq S$. Montrons que S' définit par $S' = (S \setminus eff^-(e)) \cup eff^+(e)$ appartenant par hypothèse à $\mathbb{S}^*$ appartient aussi à $\mathbb{S}$. Lorsque le processus T traite S il teste l'ensemble des événements possibles pour tenter de les appliquer dans S. L'événement $\{e\}$ est applicable dans S donc T va construire l'état $(S \setminus eff^-(e)) \cup eff^+(e) = S'$ et va l'ajouter à $\mathbb{S}$.

  On a donc montré par induction que $\mathbb{S}^* \subseteq \mathbb{S}$

On peut en conclure que $\mathbb{S}^* = \mathbb{S}$. Montrons maintenant que $\tau = \tau^*$.

Comme $\mathbb{S}^* = \mathbb{S}$ on peut réécrire τ^* de la façon suivante :

$$\tau^* = \{(S, \{e\}, S') \mid S \in \mathbb{S}, \{e\} \in 2^{\mathbb{E}(\mathcal{P})}, pre(e) \subseteq S \text{ et } S' = (S \setminus eff^-(e)) \cup eff^+(e)\}$$
$$\cup \{(S, \emptyset, S) \mid S \in \mathbb{S}, \mathcal{G} \subseteq S\}$$

Pour construire τ le processus T, pour tout $S, S' \in \mathbb{S}$ tel qu'il existe $\{e\} \in 2^{\mathbb{E}(\mathcal{P})}$ et $S' = (S \setminus eff^-(e)) \cup eff^+(e)$ ajoute la transition $(S, \{e\}, S')$ à τ. De plus pour tout $S \in \mathbb{S}$ tel que $\mathcal{G} \subseteq S$,

T ajoute $(S, \{e\}, S')$ à τ. Formellement, on a :

$$\tau = \{(S, \{e\}, S') \mid S \in \mathbb{S}, \{e\} \in 2^{\mathbb{E}(\mathcal{P})}, pre(e) \subseteq S \text{ et } S' = (S \setminus eff^-(e)) \cup eff^+(e)\}$$
$$\cup \{(S, \emptyset, S) \mid S \in \mathbb{S}, \mathcal{G} \subseteq S\} = \tau^*$$

On a montré que $T(\mathcal{P}) = \langle \mathbb{S}^*, \tau^* \rangle$ qui est un LTS, donc notre algorithme est correct.

E.2 Terminaison

Le processus T est basé sur un algorithme BFS dont l'espace d'exploration est fini et borné par le nombre d'états possible égal à $2^{|\mathbb{F}(\mathcal{P})|}$.

E.3 Complétude

Nous souhaitons montrer que notre algorithme est complet sur le sous-ensemble de LTS obtenables avec notre parseur (pas de nœud isolé, un événement unique par transition et au moins une transition réflexive). Soit un LTS défini par $\langle \mathbb{S}, \tau \rangle$, tel que $\mathbb{S} \subseteq 2^{\mathbb{F}}$ et $\tau \subseteq \mathbb{S} \times 2^{\mathbb{E}} \times \mathbb{S}$, $S_0 \in \mathbb{S}$. Il est possible d'y associé un domaine et un problème PDDL. Pour cela, on fixe :

- $\mathcal{O} = \{o\}$ on définit un unique objet pour tous les problèmes associés (nb : on pourrait ne pas en mettre du tout, mais les prédicats ne porteraient plus sur rien et notre parseur ne gère pas les actions sans paramètres).
- $Pred = \{f(o) \mid \forall f \in \mathbb{F}\}$: L'ensemble des prédicats ont une arité d'un. L'unique objet/paramètre ne représente rien, mais sert de "levier" pour dire si un prédicat est vrai ou faux.
- Pour chaque transition $(S, E, S') \in \tau$ on construit l'action associée $a \in \mathcal{A}$ de la façon suivante :
  - $param(a) = o$
  - $pre(a) = \{f(o) \mid \forall f \in S\} \cup \{notf(o) \mid \forall f \notin S\}$. On explicite la négation des fluents absents de l'état de départ pour que chaque action soit uniquement applicable entre les deux états de la transition et nulle part ailleurs.
  - $eff^+(a) = \neg S \cap S'$ avec $\neg S = \{\neg s \mid s \in S\}$
  - $eff^-(a) = S \cap \neg S'$ avec $\neg S' = \{\neg s \mid s \in S'\}$
- $S_{init} = \{f(o) \mid f \in S_0\}$
- $\mathcal{G}$ est construite à partir de $\mathbb{A} = \{S \mid S \in \mathbb{S}, (S, \varnothing, S) \in \tau\}$. La formule $\mathcal{G}$ corresponds à l'union des états appartenant à $\mathbb{A}$.

Nous avons démontré la complétude au sens formelle. Il existe pour les LTS obtenables par notre parseur une possibilité de les reconvertir en problème PDDL. Cependant, il est à noter que sur le plan computationnel la correspondance est inapplicable avec un solveur PDDL. En effet, nous restreignons l'application d'une action à une unique état en générant des préconditions de taille $|\mathcal{F}|$ ce qui fait exploser la combinatoire du problème et perdre l'intérêt du solveur.

nb : on ne montre pas vraiment la complétude, mais plutôt l'idée de représentabilité en PDDL. On ne perd pas en information sémantiquement, mais cette transition inverse est peu optimale. Une autre possibilité de démonstration intéressante serait de construire un PDDL moins restreint et plus compact. En contrepartie des états non présents dans le LTS de base pourrait être atteints en appliquant notre parseur au PDDL construit. Dans tous les cas, pour notre parseur actuel, il n'existe pas d'application inverse qui reconstruise à l'identique le problème PDDL qui a permis d'obtenir un LTS.

F PDDL

F.1 Domaine Block World classique

```
(define (domain bw)
  (:requirements :strips)
  (:constants red green blue)
  (:predicates
      (on ?x ?y)       ; object ?x is on ?object ?y
      (on-table ?x)    ; ?x is directly on the table
      (clear ?x)       ; ?x has nothing on it
      (arm-empty)      ; robot isn't holding anything
      (holding ?x))    ; robot is holding ?x

  ;; les quatres actions classique de manipulation d'objet
  (:action pick-up
     :parameters (?ob1)
     :precondition (and (clear ?ob1) (on-table ?ob1) (arm-empty))
     :effect
     (and (not (on-table ?ob1))
             (not (clear ?ob1))
             (not (arm-empty))
             (holding ?ob1)))

  (:action put-down
     :parameters (?ob)
     :precondition (holding ?ob)
     :effect
     (and (not (holding ?ob))
           (clear ?ob)
           (arm-empty)
           (on-table ?ob)))

  (:action stack
     :parameters (?sob ?sunderob)
     :precondition (and (holding ?sob) (clear ?sunderob))
     :effect
```

```
      (and (not (holding ?sob))
            (not (clear ?sunderob))
            (clear ?sob)
            (arm-empty)
            (on ?sob ?sunderob)))

  (:action unstack
     :parameters (?sob ?sunderob)
     :precondition (and (on ?sob ?sunderob) (clear ?sob) (arm-empty))
     :effect
     (and (holding ?sob)
            (clear ?sunderob)
            (not (clear ?sob))
            (not (arm-empty))
            (not (on ?sob ?sunderob))))
)
```

F.2 Domaine et problème Poison

Les fichiers PDDL permettant d'obtenir le LTS de la section 5.4.

```
(define (domain poison)

  (:requirements :strips :negative-preconditions :disjunctive-preconditions)

  (:predicates
    (ok ?v)
    (emp ?v)
    (anti ?v)
    (vide ?v)
    (mort ?v)
    (vivant ?v)
  )
  (:action boire_vo
    :parameters (?v)
    :precondition    (and (ok ?v) (not (emp ?v))
                      (not (anti ?v)) (not (vide ?v))
                      (not (mort ?v)) (not (vivant ?v)))
    :effect (and (not (ok ?v))
                 (vivant ?v))
  )

  (:action emp
    :parameters (?v)
    :precondition   (and (ok ?v) (not (emp ?v))
                         (not (anti ?v)) (not (vide ?v))
```

```
                        (not (mort ?v)) (not (vivant ?v)))
  :effect (and
            (not (ok ?v))
            (emp ?v))
)

(:action vider_vo
  :parameters (?v)
  :precondition   (and (ok ?v) (not (emp ?v))
                       (not (anti ?v)) (not (vide ?v))
                       (not (mort ?v)) (not (vivant ?v)))
  :effect (and
            (not (ok ?v))
            (vide ?v))
)

(:action anti
  :parameters (?v)
  :precondition   (and (not (ok ?v)) (emp ?v)
                       (not (anti ?v)) (not (vide ?v))
                       (not (mort ?v)) (not (vivant ?v)))

  :effect (and
            (not (emp ?v))
            (anti ?v))
)

(:action boire_ve
  :parameters (?v)
  :precondition   (and (not (ok ?v)) (emp ?v)
                       (not (anti ?v)) (not (vide ?v))
                       (not (mort ?v)) (not (vivant ?v)))

  :effect (and (not(emp ?v)) (mort ?v))
)

(:action vider_ve
  :parameters (?v)
  :precondition   (and (not (ok ?v)) (emp ?v)
                       (not (anti ?v)) (not (vide ?v))
                       (not (mort ?v)) (not (vivant ?v)))
  :effect (and (not (emp ?v)) (vide ?v))
)

(:action rien
  :parameters (?v)
  :precondition   (and (not (ok ?v)) (not (emp ?v))
```

```
                        (not (anti ?v)) (vide ?v)
                        (not (mort ?v)) (not (vivant ?v)))

    :effect (and (not (vide ?v)) (mort ?v))
  )

  (:action boire_va
    :parameters (?v)
    :precondition   (and (not (ok ?v)) (not (emp ?v))
                        (anti ?v) (not (vide ?v))
                        (not (mort ?v)) (not (vivant ?v)))

    :effect (and (not (anti ?v)) (vivant ?v))
  )

)


(define (problem poison_p0)
    (:domain poison)
    (:objects V)

    (:init (ok V))

    (:goal (or (vivant V) (mort V))))
```

Références

Henne, Paul et al. (juill. 2021). « Counterfactual thinking and recency effects in causal judgment ». In : *Cognition* 212, p. 104708. ISSN : 0010-0277. DOI : 10.1016/j.cognition.2021.104708. URL : https://www.sciencedirect.com/science/article/pii/S001002772100127X (visité le 07/05/2026).

Icard, Thomas F., Jonathan F. Kominsky et Joshua Knobe (2017). « Normality and actual causal strength ». In : *Cognition*. DOI : 10.1016/j.cognition.2017.01.010. URL : https://sci-hub.fr/10.1016/j.cognition.2017.01.010 (visité le 10/05/2026).

Kominsky, Jonathan F. et al. (avr. 2015). « Causal superseding ». In : *Cognition* 137, p. 196-209. ISSN : 0010-0277. DOI : 10.1016/j.cognition.2015.01.013. URL : https://www.sciencedirect.com/science/article/pii/S0010027715000220 (visité le 05/05/2026).

McGill, Ann L. et Ann E. Tenbrunsel (2000). « Mutability and propensity in causal selection. » In : *Journal of Personality and Social Psychology*. DOI : 10.1037/0022-3514.79.5.677. URL : https://sci-hub.fr/10.1037/0022-3514.79.5.677 (visité le 02/06/2026).

Sarmiento, Camilo (2025). « Formalising Overdetermination in a Labelled Transition System ». en. In.